DAY 1

Module 1

From Vibe Coding to Spec-Driven Development

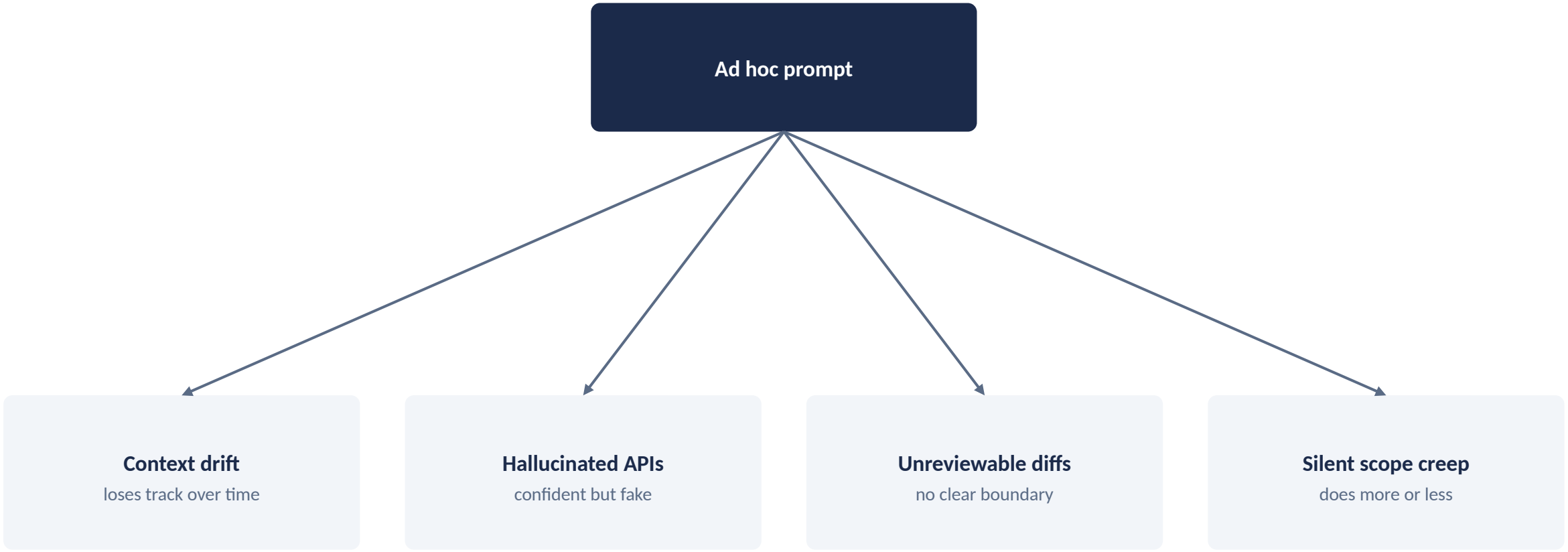
Why unstructured AI prompting breaks down, and what spec-driven development changes.

What You'll Learn

1. Recognize the failure modes of unstructured AI prompting
2. Explain the core thesis of spec-driven development
3. Locate GitHub Spec Kit and OpenSpec within the wider SDD landscape
4. Scaffold both frameworks side by side in your own project

THE PROBLEM

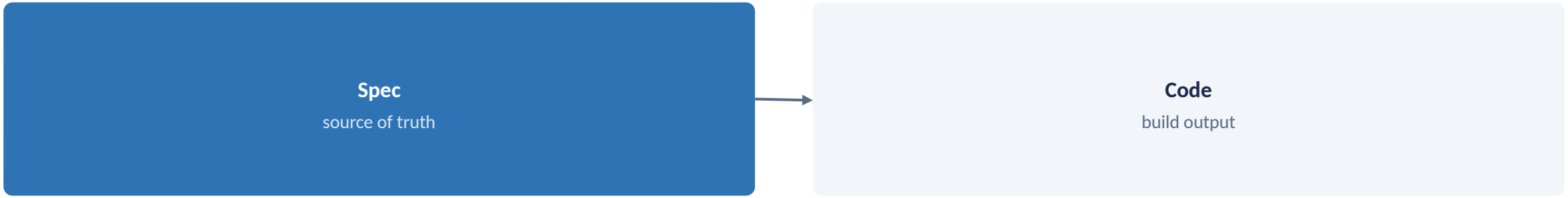
Failure Modes of Ad Hoc Prompting



Four ways unstructured prompting quietly goes wrong

THE FIX

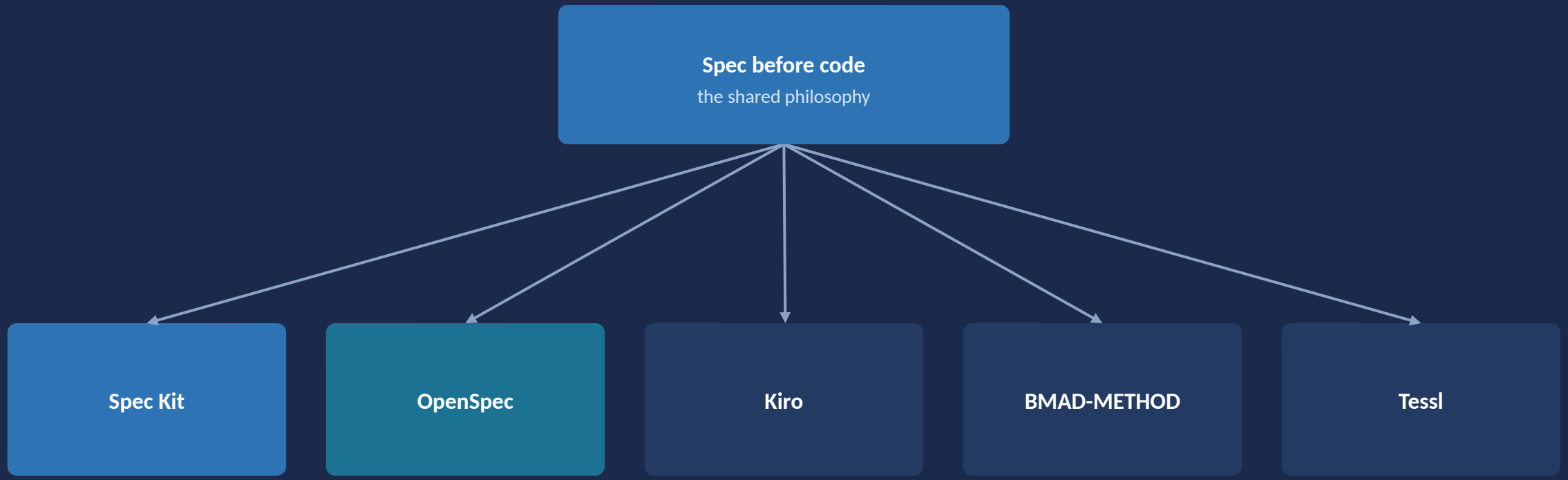
The Core Thesis of Spec-Driven Development



The spec is written first; code is generated from it, the way a compiler turns source into a binary

LANDSCAPE

Multiple frameworks implement the same core thesis, differently.



This course uses Spec Kit hands-on, with OpenSpec alongside for comparison

APPLYING THE CONCEPTS

GitHub Spec Kit

SPEC KIT

Scaffolding a Project

```
specify init
```

Scaffolds a new project: creates .specify/ with templates, scripts, and agent-specific command files. Use --integration copilot or --integration claude to target your agent, so the generated commands match the tool you'll actually run them in.

The map: The full SDD lifecycle this course follows: constitution, specify, clarify, plan, tasks, checklist, analyze, implement. This is the map for the rest of the course, so every later module is really a deep dive on one step of this same sequence.

Not all mandatory: Only specify is strictly required before plan. Clarify, checklist, and analyze are quality gates you add for anything with meaningful ambiguity, so simple, low-risk changes don't have to pay for gates they don't need.

APPLYING THE CONCEPTS

OpenSpec

OPENSPEC

Scaffolding a Project

```
openspec init
```

Scaffolds openspec/ with specs/ (source of truth), changes/ (proposed updates), and config.yaml. Install first with npm install -g @fission-ai/openspec, since the CLI has to exist locally before it can scaffold anything.

The default loop: Default quick path: explore (optional), propose, apply, sync (optional), archive, so a small change can move through in four commands while a riskier one can add explore and sync for extra checkpoints.

Different spec model: Where Spec Kit produces one spec file per feature, OpenSpec proposes changes as compact deltas (ADDED, MODIFIED, REMOVED) against a single living spec, which is why OpenSpec's spec directory stays smaller over time even as the project grows.

Module 1 at a Glance

	GitHub Spec Kit	OpenSpec
Install	specify init --integration <agent>	npm i -g @fission-ai/openspec + openspec init
Creates	.specify/ (templates, scripts, agent files)	openspec/ (specs/, changes/, config.yaml)
Spec model	One file per feature	Single living spec + deltas

Key Takeaways

- Ad hoc prompting fails in predictable, nameable ways, which means those failures are also predictably preventable.
- SDD's core move: make the spec the source of truth, not an afterthought written up after the code already exists.
- You now have both frameworks scaffolded and ready to use for the rest of the course, so every later hands-on exercise builds on this same setup.